

数字的奥秘

高精度、位运算与离散化 · 7题 · C++ & Python 双语对照

NQ122 高精度加法 AcWing 791

给定两个正整数，计算它们的和。数据范围： $1 \leq \text{整数长度} \leq 100000$

输入

共两行，每行包含一个整数。

输出

共一行，包含所求的和。

来源

AcWing 791

输入

12
23

输出

35

提示 [原题链接](#) [参考题解](#) [Y总讲解](#) [Y总代码](#)

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6+7;

vector<int> add(vector<int> &A, vector<int> &B){
    vector<int> C;
    int t=0;
    for (int i=0; i<A.size()||i<B.size(); i++) {
        if (i<A.size()) t+=A[i];
        if (i<B.size()) t+=B[i];
        //对t取模, 取进位
        C.push_back(t %10); //把t模10的余数入数组, 商为进位
        t /= 10; //算出进位, 供下一次循环使用
    }
    if(t) C.push_back(1); //最高位进位
    return C;
}

int main(){
    //输入为字符串
    string a,b;
    vector<int> A,B;

    cin >> a>>b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for(int i= a.size()-1; i>=0; i--) A.push_back(a[i]-
'0');
    for(int i= b.size()-1; i>=0; i--) B.push_back(b[i]-
'0');

    auto C = add(A,B);

    for(int i= C.size()-1; i>=0; i--) cout<<C[i]; //打印C
    return 0;
}

```

Python

```

# AcWing 791
# Python solution

```

NQ123 高精度减法 AcWing 792

给定两个正整数，计算它们的差，计算结果可能为负数。1≤整数长度≤100000

输入 共两行，每行包含一个整数。

输出 共一行，包含所求的差。

来源 AcWing 792

输入	输出
32	21
11	

提示 原题链接 Y总讲解 参考题解 Y总代码

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6 + 7;
bool isGreater(vector<int> &A, vector<int> &B)
{
    if (A.size() != B.size()) //比较大小
        return A.size() > B.size();
    //否则大小相等，逐位比较大小
    for (int i = A.size() - 1; i >= 0; i--)
        if (A[i] != B[i])
            return A[i] > B[i];
    return true; //每个数字都相同，长度相同，两数相等
}
//C= A-B
vector<int> sub(vector<int> &A, vector<int> &B)
{
    vector<int> C;
    //A是比较大的数
    int t = 0; //第一轮减法进位为0
    for (int i = 0; i < A.size(); i++)
    {
        t = A[i] - t;
        if (i < B.size())
            t -= B[i]; //A[i]-B[i] + t
        C.push_back((t + 10) % 10); //得到对应位
        if (t < 0)
            t = 1;
        else
            t = 0; //判断是否借一位
    }
    //去掉开头的0
    while (C.size() > 1 && C.back() == 0) C.pop_back();

    return C;
}

int main()
{
    //输入为字符串
    string a, b;
    vector<int> A, B;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');
    for (int i = b.size() - 1; i >= 0; i--)
        B.push_back(b[i] - '0');
    //判断A与B的大小
    if (isGreater(A, B))
    {
        auto C = sub(A, B);
        for (int i = C.size() - 1; i >= 0; i--)
            cout << C[i]; //打印C
    }
    else
    {
        auto C = sub(B, A);
        cout << "-";
        for (int i = C.size() - 1; i >= 0; i--)
            cout << C[i]; //打印C
    }
}

```

Python

```

# AcWing 792
# Python solution

```

```

    }
    return 0;
}

```

NQ124 二进制中1的个数 AcWing 801

给定一个长度为 n 的数列，请你求出数列中每个数的二进制表示中1的个数。数据范围： $1 \leq n \leq 100000, 0 \leq \text{数列中元素的值} \leq 10^9$

输入 第一行包含整数 n 。第二行包含 n 个整数，表示整个数列。

输出 共一行，包含 n 个整数，其中的第 i 个数表示数列中的第 i 个数的二进制表示中1的个数。

来源 AcWing 801

输入

5
1 2 3 4 5

输出

1 1 2 1 2

C++

```

#include <iostream>
using namespace std;
inline int lowbit(int &x)
{
    return x & -x; //x的二进制的最后一个1所构成的整数
}
int main()
{
    int n, x;
    cin >> n;
    while (n--)
    {
        //读入一个数，并且计算1的个数，并且输出
        cin >> x;
        int res = 0; //计数变量
        while (x)
            x -= lowbit(x), res++; //使用表达式，简化代码去
        cout << res << " "; //输出运算结果
    }
    return 0;
}

```

Python

```

# AcWing 801
# Python solution

```

NQ125 高精度乘法 AcWing 793

给定两个正整数 A 和 B ，请你计算 $A * B$ 的值。 $1 \leq A$ 的长度 $\leq 100000, 0 \leq B \leq 10000$

输入 共两行，第一行包含整数 A ，第二行包含整数 B 。

输出 共一行，包含 $A * B$ 的值。

来源 AcWing 793

输入

2
3

输出

6

提示 原题链接 Y总讲解 参考题解 Y总代码

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6 + 7;

//C= A*b,t的用法很精彩!
vector<int> mul(vector<int> &A, int b){
    vector<int> C;
    int t=0;
    for(int i=0; i<A.size() || t; i++){
        //如果t还有剩余或者A还有剩余,乘法结果C需要加新的位
        if(i<A.size()) t+=A[i]*b;//把b当作整体做乘法
        C.push_back(t % 10);//存储第i位
        t /=10; //更新t
    }
    while(C.size()>1 && C.back()==0) C.pop_back();//去掉C
    开头的0
    return C;
}

int main()
{
    //输入为字符串
    string a;
    int b;//输入的b比较小,作为一个整体处理
    vector<int> A,C;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');
    C=mul(A,b);
    for (int i = C.size() - 1; i >= 0; i--)
        cout << C[i]; //打印C
    return 0;
}

```

Python

```

# AcWing 793
# Python solution

```

NQ126 高精度除法 AcWing 794

给定两个非负整数A, B, 请你计算 A / B 的商和余数。数据范围: $1 \leq A \text{ 的长度} \leq 100000, 1 \leq B \leq 10000$ B 一定不为0

输入 共两行, 第一行包含整数A, 第二行包含整数B。

输出 共两行, 第一行输出所求的商, 第二行输出所求余数。

来源 AcWing 794

输入

7
2

输出

3
1

提示 原题链接 Y总讲解 参考题解 Y总代码

C++

```

#include <iostream>
#include <cstring>
#include <vector>
#include <algorithm>

using namespace std;
const int N = 1e6 + 7;

//C= A/b 余数为r
vector<int> div(vector<int> &A, int b, int &r){
    vector<int> C;
    r=0;//余数
    for(int i=A.size()-1; i>=0; i--){
        r = r* 10 + A[i]; //从最高位开始, 当前余数r需要上一次运
        算的r乘10
        C.push_back(r/b);
        r %= b;
    }
    reverse(C.begin(), C.end()); //定义的数是最低位在C[0], 因此
    需要取逆
    while(C.size(>1 && C.back()==0) C.pop_back(); //去掉C
    开头的0字符
    return C;
}

int main()
{
    //输入为字符串
    string a;
    int b; //输入的b比较小, 作为一个整体处理
    vector<int> A;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');

    int r;
    auto C=div(A,b,r); //使用auto自动辨别变量类型

    for (int i = C.size() - 1; i >= 0; i--)
        cout << C[i]; //打印C
        cout<<endl<<r<<endl; //打印余数
    return 0;
}

```

Python

```

# AcWing 794
# Python solution

```

假定有一个无限长的数轴，数轴上每个坐标上的数都是0。现在，我们首先进行 n 次操作，每次操作将某一位置 x 上的数加 c 。接下来，进行 m 次询问，每个询问包含两个整数 l 和 r ，你需要求出在区间 $[l, r]$ 之间的所有数的和。

输入

第一行包含两个整数 n 和 m 。接下来 n 行，每行包含两个整数 x 和 c 。再接下里 m 行，每行包含两个整数 l 和 r 。

输出

共 m 行，每行输出一个询问中所求的区间内数字和。

来源

AcWing 802

输入	输出
3 3	8
1 2	0
3 6	5
7 5	
1 3	
4 6	
7 8	

提示 原题

C++

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
typedef pair<int, int> PII; //自定义类型PII, 表示容器类型是<int, int>的pair

const int N = 300007; //需要记录n个坐标值, 以及m个区间的坐标, 所以是100000*(1+2)
int n, m;
int a[N], s[N];

vector<int> alls; //存储所有的点坐标, 包括询问点坐标
vector<PII> add, query; //输入操作的坐标, 查询的区间坐标
//在alls之中, 查找值是x的元素值的下标, 并且映射到[1...n]
int find(int x)
{
    int left = 0, right = alls.size() - 1; //vector从0开始计数
    while (left < right)
    {
        int mid = left + right >> 1; //取中值
        if (alls[mid] >= x)
            right = mid;
        else
            left = mid + 1;
    }
    return right + 1; //把坐标值映射到1, 2, 3...而不是从0开始
}

///? 使用离散化的方式来预处理输入数据
///? 把输入的数字映射成{下标, 数字}的pair, 然后再用差分的方法计算区间和
///? 本题目可以参考797. 差分的模板写成
int main()
{
    cin >> n >> m;
    ///!离散化: 构造映射表

    for (int i = 0; i < n; i++)
    {
        ///todo 不直接把数存入a[i], 乃是存入add容器内
        int x, c;
        cin >> x >> c;
        add.push_back({x, c}); ///?将某一位置x上的数加c
        alls.push_back(x); //把x位置加入alls
    }
    // 读入query
    for (int i = 0; i < m; i++)
    {
        int l, r; //区间l和r
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l); //坐标与alls的下标做映射
        alls.push_back(r);
    }
    ///alls是下标映射表, 需要去重
    ///使用STL算法库的sort和unique
    sort(alls.begin(), alls.end());
    ///unique返回不重复区间的下一个元素位置, 用erase去掉尾部
    alls.erase(unique(alls.begin(), alls.end()), alls.end());
}

```

Python

```

# AcWing 802
# Python solution

```



```

//针对离散化的数据, 计算区间和
for (auto it : add) //用c++11语法遍历add vector
{
    //初始化a[i],用离散化后的坐标
    int index = find(it.first);
    a[index] += it.second; //将index位置上的数加c
}
//预处理前缀和
for (int i = 1; i <= alls.size(); i++)
    s[i] = s[i - 1] + a[i];
//处理询问, 并且输出
for (auto item : query)
{
    int l = find(item.first), r = find(item.second);
    cout << s[r] - s[l - 1] << endl; //前缀和计算公式
}
return 0;
}

```

NQ128 区间合并 AcWing 803

给定 n 个区间 $[l_i, r_i]$, 要求合并所有有交集的区间。注意如果在端点处相交, 也算有交集。输出合并完成后的区间个数。例如: $[1,3]$ 和 $[2,6]$ 可以合并为一个区间 $[1,6]$ 。数据范围: $1 \leq n \leq 100000$, $-10^9 \leq l_i \leq r_i \leq 10^9$

输入 第一行包含整数 n 。接下来 n 行, 每行包含两个整数 l 和 r 。

输出 共一行, 包含一个整数, 表示合并区间完成后的区间个数。

来源 AcWing 803

输入	输出
5	3
1 2	
2 4	
5 6	
7 8	
7 9	

提示 Y总讲解 参考题解

C++

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
typedef pair<int, int> PII; //自定义类型PII, 表示容器类型是<int, int>的pair

const int N = 100007; //1<=n<=100000
int n; //n个区间

vector<PII> segments; //区间坐标

void merge(vector<PII> &segs)
{
    vector<PII> res;
    //pair排序先按左端点, 后右端点
    sort(segs.begin(), segs.end());

    //数字范围是-10^9<num<10^9, 因此负无穷大初始化为-2e9
    int left = -2e9, right = -2e9; //当前操作的区间[left, right]

    for (auto seg : segs) //遍历segs
        if (right < seg.first) //区间没有交集, 更新当前区间
        {
            //存储合并完毕的区间[left, right]
            if (left != -2e9) res.push_back({left, right});

            //更新当前操作区间
            left = seg.first, right = seg.second;
        }
        else right = max(right, seg.second); //更新右端点, 扩展区间

    if (left != -2e9) res.push_back({left, right}); //把最后一个区间入栈

    segs = res; //更新segs
}

int main()
{
    cin >> n;
    // 读入n组区间
    for (int i = 0; i < n; i++)
    {
        int l, r; //区间l和r
        cin >> l >> r;
        segments.push_back({l, r});
    }
    //合并区间segment
    merge(segments);
    //输出合并后的区间个数
    cout<<segments.size()<<endl; //输出合并后的区间元素个数

    return 0;
}

```

Python

```

# AcWing 803
# Python solution

```